

RAPPORT D'ANALYSE STATIQUE

Application Android — OWASP UnCrackable Level 1

Titre	Analyse statique de l'application UnCrackable Level 1
Date de l'analyse	Avril 2025
Analyste	Benchelha Hidaya
APK	UnCrackable-Level1.apk — version 1.0 (versionCode 1)
Provenance	OWASP Mobile Security Testing Guide (MSTG) — dépôt officiel GitHub
Outils utilisés	JADX-GUI v1.5.x, dex2jar v2.x, JD-GUI v1.6.x

B) Résumé exécutif

Cette analyse statique a révélé 1 vulnérabilité potentielle de niveau moyen et 7 observations de niveau faible dans l'application UnCrackable Level 1.

Principales préoccupations : La configuration `android:allowBackup="true"` constitue la principale préoccupation de sécurité. Aucune donnée sensible (mot de passe, token, URL) n'a été découverte directement exposée. L'application ne possède ni permission réseau ni attribut de débogage actif.

Niveau de risque global : FAIBLE — L'application présente un profil de risque globalement bas dans le cadre d'une analyse statique.

Actions prioritaires recommandées :

1. Désactiver `android:allowBackup="true"` ou chiffrer les données locales sensibles.
2. Effectuer une analyse dynamique complémentaire pour valider l'absence de fuite de données à l'exécution.
3. Approfondir l'analyse de la logique de vérification de clé secrète identifiée dans la classe `sg.vantagepoint.a.a`.

C) Constats détaillés

Constat #1 — Présence de `android:allowBackup="true"`

Sévérité : Moyenne

Description : L'attribut `android:allowBackup="true"` est présent dans la balise `<application>` du manifeste Android. Cette configuration autorise le système Android à sauvegarder les données applicatives via adb backup, ce qui peut exposer des informations sensibles stockées localement si elles ne sont pas chiffrées.

Localisation : `AndroidManifest.xml` → balise `<application>`

Impact potentiel : Un attaquant ayant un accès physique ou via USB au dispositif pourrait extraire une sauvegarde complète des données de l'application, y compris des fichiers, préférences et bases de données.

Remédiation recommandée : Définir `android:allowBackup="false"` ou, si la sauvegarde est nécessaire, utiliser `android:fullBackupContent` pour exclure les fichiers sensibles.

Constat #2 — Logique de vérification de secret dans `sg.vantagepoint.a.a`

Sévérité : Élevée (potentielle — décompilation)

Description : La classe `sg.vantagepoint.a.a` contient une logique de vérification cryptographique. Le code décompilé révèle l'utilisation d'opérations AES/ECB et d'une clé construite à partir de composants internes. Bien que l'obfuscation complique la lecture, la logique de validation du secret est visible dans le code décompilé.

Localisation : `Package sg.vantagepoint.a` → classe `a.java` (méthodes de vérification de clé secrète)

Impact potentiel : Un analyste peut potentiellement reconstruire le secret attendu par rétro-ingénierie statique ou dynamique, contournant ainsi les mécanismes de protection de l'application.

Remédiation recommandée : Utiliser des techniques de protection avancées : obfuscation renforcée, stockage de secrets dans un environnement de confiance (Android Keystore), ou vérification côté serveur.

Constat #3 — Aucune permission réseau déclarée

Sévérité : Informative (point positif)

Description : L'application ne déclare aucune permission android.permission.INTERNET dans son manifeste, ce qui réduit la surface d'attaque liée aux communications réseau.

Localisation : AndroidManifest.xml → section <uses-permission>

Impact potentiel : Aucun (positif). L'absence de permissions réseau confirme que l'application fonctionne en mode hors ligne, sans communication avec des serveurs externes.

Constat #4 — Attributs sensibles absents du manifeste

Sévérité : Informative (point positif)

Description : Les attributs android:debuggable="true" et android:usesCleartextTraffic="true" sont absents du manifeste. L'application n'est donc pas déployée en mode debug et ne force pas l'utilisation du trafic HTTP non chiffré.

Localisation : AndroidManifest.xml → balise <application>

Impact potentiel : Aucun risque immédiat. Ces absences sont des bonnes pratiques de sécurité respectées.

A) Informations générales — Analyse du manifeste

1. Informations générales de l'application

Package principal	owasp.mstg.uncrackable1
versionCode	1
versionName	1.0
minSdkVersion	19 (Android 4.4 KitKat)
targetSdkVersion	28 (Android 9 Pie)

2. Permissions demandées

Dans ce manifeste, aucune permission n'est déclarée avec la balise `<uses-permission />`. L'application ne demande aucune permission sensible (Internet, stockage, caméra, localisation, SMS, contacts, etc.).

Conclusion : Aucune permission Android dangereuse ou sensible n'est demandée dans le manifeste.

3. Composants déclarés

Activities

Une seule activité est déclarée :

Type	Activity
Nom du composant	sg.vantagepoint.uncrackable1.MainActivity
Rôle	Activité principale de l'application

Aucun Service, Broadcast Receiver ou Content Provider n'est déclaré dans le manifeste.

4. Composants exportés

L'activité principale contient un intent-filter MAIN/LAUNCHER. Bien que `android:exported="true"` ne soit pas explicitement présent, la présence d'un intent-filter rend cette activité accessible comme point d'entrée de l'application.

Composant	Intent-filter	Exporté explicitement	Remarque
sg.vantagepoint.uncrackable1.MainActivity	Oui	Non	Activité launcher — point d'entrée public de l'application

5. Configurations sensibles

Attribut	Statut	Niveau de risque
android:usesCleartextTraffic="true"	Absent	Aucun risque
android:debuggable="true"	Absent	Aucun risque

android:allowBackup="true"	Présent	Risque moyen — données potentiellement extractibles
----------------------------	---------	---

6. Observations documentées — Analyse des patterns critiques

N°	Observation	Emplacement	Risque	Commentaire
1	Présence du mot-clé http	AndroidManifest.xml	Faible	Namespace XML Android (http://schemas.android.com/apk/res/android). Pas une URL réseau réelle.
2	Présence du mot-clé http	res/layout/activity_main.xml	Faible	Namespace Android dans le fichier layout. Aucun serveur HTTP externe identifié.
3	Présence du mot-clé http	res/menu/menu_main.xml	Faible	Déclaration XML du menu Android. Aucun trafic clair ou endpoint réseau observé.
4	Recherche du pattern pwd	Recherche globale JADX	Faible	Aucun résultat trouvé. Aucun mot-clé lié à un mot de passe n'a été détecté.
5	Absence de permission réseau	AndroidManifest.xml	Faible	Aucune permission android.permission.INTERNET déclarée. Risque de communication externe réduit.
6	Absence usesCleartextTraffic	AndroidManifest.xml	Faible	L'application ne déclare pas l'autorisation du trafic HTTP non chiffré.
7	Absence de debuggable="true"	AndroidManifest.xml	Faible	L'application n'est pas déclarée comme débogable dans le manifeste.
8	Présence de allowBackup="true"	AndroidManifest.xml, balise <application>	Moyen	Peut permettre la sauvegarde des données applicatives. Risque si des données sensibles sont stockées localement.

Conclusion de l'analyse des patterns critiques

Les recherches effectuées dans JADX n'ont pas révélé de mot de passe, token, clé secrète ou URL sensible directement exposée dans les ressources de l'application. Les seules occurrences du mot-clé http correspondent aux namespaces XML standards d'Android (AndroidManifest.xml, activity_main.xml, menu_main.xml) et ne représentent pas une communication réseau réelle.

Aucune permission sensible n'est demandée, et les attributs android:debuggable="true" et android:usesCleartextTraffic="true" sont absents. Cependant, la présence de android:allowBackup="true" constitue une configuration à surveiller.

Check final — Analyse statique

Élément demandé	Statut / Résultat
Recherches effectuées sur les patterns critiques	Oui
Au moins 5 observations documentées	Oui — 8 observations
Niveau de risque évalué pour chaque observation	Oui
Emplacements précis notés	Oui

Découvertes sensibles identifiées	Aucune découverte critique
Configuration sensible notée	Oui : android:allowBackup="true"

Comparaison JADX-GUI vs JD-GUI

Classe analysée

Pour comparer les deux outils, la même application APK a été analysée avec JADX-GUI et JD-GUI.

Classe principale observée	sg.vantagepoint.uncrackable1.MainActivity
Classe logique interne	sg.vantagepoint.a.a
Fichier utilisé (JD-GUI)	app.jar (obtenu via dex2jar depuis classes.dex)

Tableau comparatif

Critère comparé	JADX-GUI	JD-GUI	Observation
Navigation & organisation	Structure proche de l'APK Android avec packages, manifeste et ressources	Présente surtout les classes Java du fichier .jar	JADX-GUI plus adapté à l'analyse complète
Lisibilité du code	Code plus lisible et mieux reconstruit dans le contexte Android	Code parfois moins clair après conversion dex → jar	JADX-GUI donne une meilleure lecture du code Android
Ressources Android	Affiche AndroidManifest.xml, strings.xml, layouts, menus, etc.	N'affiche pas correctement les ressources Android	JD-GUI est limité pour les ressources
Gestion des constructions	Reconnaît Activities, layouts et ressources spécifiques Android	Se concentre sur les classes Java décompilées	JADX-GUI plus complet pour comprendre l'application
Impact obfuscation	Navigation possible même avec classes obfusquées (a, b, c...)	Affiche aussi les classes obfusquées, contexte moins clair	JADX aide mieux à comprendre les relations entre fichiers
Méthodes & classes Java	Lecture directe depuis DEX/APK	Lecture des méthodes Java après conversion en .jar	JD-GUI utile pour vérifier le code Java reconstruit

Différences notables documentées

1. Affichage des ressources Android

Dans JADX-GUI, il est possible d'explorer directement les fichiers Android : AndroidManifest.xml, res/values/strings.xml, res/layout/activity_main.xml, res/menu/menu_main.xml. Cela permet d'analyser les permissions, composants et chaînes de caractères.

Dans JD-GUI, ces ressources ne sont pas affichées de manière complète, car l'outil travaille principalement sur les fichiers .class du fichier .jar.

Conclusion : JADX-GUI est plus adapté pour analyser les ressources Android d'un APK.

2. Navigation dans la structure de l'application

Dans JADX-GUI, la navigation est organisée autour de la structure réelle de l'APK. On peut facilement passer du manifeste aux classes Java et aux ressources.

Dans JD-GUI, la navigation se limite principalement aux packages Java (sg.vantagepoint.a, sg.vantagepoint.uncrackable1) avec les classes MainActivity.class, a.class, b.class, c.class.

Conclusion : JD-GUI permet de lire les classes Java, mais JADX-GUI donne une vue plus complète de l'application Android.

3. Lisibilité du code décompilé

Dans JD-GUI, certaines classes apparaissent avec des noms courts ou obfusqués (a.class, b.class, c.class). Le code reste lisible, mais difficile à contextualiser.

Dans JADX-GUI, le code est mieux présenté avec une navigation plus pratique entre méthodes, appels et ressources.

Conclusion : JADX-GUI est généralement plus lisible pour l'analyse Android.

Forces et faiblesses des outils

	JADX-GUI	JD-GUI
Forces	Analyse directe de l'APK • Affichage du manifeste et des ressources XML • Navigation claire • Très adapté à Android	Simple à utiliser • Interface légère • Bon affichage des classes Java • Utile pour lire un fichier .jar généré avec dex2jar
Faiblesses	Code décompilé parfois approximatif • Classes obfusquées difficiles • Peut être lourd à lancer	Nécessite conversion classes.dex → app.jar • Pas de vue APK complète • Ressources Android non affichées • Moins adapté au manifeste et XML

Conclusion générale — Comparaison des outils

Pour l'analyse d'une application Android, JADX-GUI est l'outil le plus adapté, car il permet d'ouvrir directement l'APK, de consulter le manifeste, les ressources XML et les classes Java dans une même interface.

JD-GUI reste utile comme outil complémentaire pour analyser le code Java après conversion du fichier classes.dex en .jar. Cependant, il est moins complet pour une analyse Android globale.

Dans ce lab : JADX-GUI est l'outil principal pour l'analyse statique globale, tandis que JD-GUI est utilisé comme outil secondaire pour comparer le code décompilé des classes Java.

Check final — Comparaison des outils

Élément demandé	Statut / Résultat
Même classe analysée dans les deux outils	Oui : sg.vantagepoint.uncrackable1.MainActivity
Comparaison avec une classe interne	Oui : sg.vantagepoint.a.a
Au moins 3 différences documentées	Oui — 3 différences majeures documentées
Forces de JADX-GUI identifiées	Oui
Faiblesses de JADX-GUI identifiées	Oui
Forces de JD-GUI identifiées	Oui
Faiblesses de JD-GUI identifiées	Oui
Conclusion sur l'outil le plus adapté	Oui : JADX-GUI pour l'analyse Android complète

D) Annexes

Annexe 1 — Liste complète des permissions

Aucune permission n'est déclarée dans le manifeste de l'application UnCrackable Level 1. La balise `<uses-permission />` est absente du fichier `AndroidManifest.xml`.

<code>android.permission.INTERNET</code>	Absente
<code>android.permission.READ_EXTERNAL_STORAGE</code>	Absente
<code>android.permission.WRITE_EXTERNAL_STORAGE</code>	Absente
<code>android.permission.CAMERA</code>	Absente
<code>android.permission.ACCESS_FINE_LOCATION</code>	Absente
<code>android.permission.READ_CONTACTS</code>	Absente
<code>android.permission.SEND_SMS</code>	Absente

Annexe 2 — Composants exportés

Un seul composant est potentiellement accessible depuis l'extérieur de l'application :

Composant	Type	Intent-filter	Remarque
<code>sg.vantagepoint.uncrackable1.MainActivity</code>	Activity	MAIN / LAUNCHER	Point d'entrée public de l'application — <code>android:exported</code> implicite

Aucun Service, Broadcast Receiver ou Content Provider exporté n'a été détecté.

Annexe 3 — Autres éléments pertinents

Fichiers XML analysés

- `AndroidManifest.xml` — manifeste de l'application
- `res/layout/activity_main.xml` — layout de l'interface principale
- `res/menu/menu_main.xml` — menu de l'application
- `res/values/strings.xml` — chaînes de caractères

Classes Java analysées

- `sg.vantagepoint.uncrackable1.MainActivity` — activité principale
- `sg.vantagepoint.a.a` — logique cryptographique interne (vérification de clé)
- `sg.vantagepoint.a.b` — classe auxiliaire (obfusquée)
- `sg.vantagepoint.a.c` — classe auxiliaire (obfusquée)

Informations sur le fichier APK

Fichier source	UnCrackable-Level1.apk
Origine	OWASP MSTG — dépôt GitHub officiel
Objectif pédagogique	Application intentionnellement vulnérable pour la formation en sécurité mobile

Format d'analyse	Analyse statique uniquement (pas d'émulateur ni de device physique)
------------------	---

— *Fin du rapport* —